COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

23IT723 - DEVOPS LABORATORY

PROJECT

CLOUD INFRASTRUCTURE MONITORING AND AUTOMATED ALERT SYSTEM



OCTOBER 2026

Submitted by

2303717620522022 JAYASREE R

COIMBATOREINSTITUTEOFTECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

BONAFIDE CERTIFICATE

Certified that this project report titled "CLOUD INFRASTRUCTURE MONITORING AND AUTOMATED ALERT SYSTEM" is the bonafide work of [STUDENT NAME] (Register No: [REGISTER NUMBER]) completed during the academic year 2025-2026 - Semester VII for the project presentation of 23IT723 - DEVOPS LABORATORY under our supervision.

Certified that the candidate was examined by using the project work examination.

Review Members:

1. **Dr.M.Sangeetha**
2. **Dr.E.Arul**

23IT723 – DEVOPS LABORATORY

FINAL MINI PROJECT – EVALUATION SCHEME

Final Mini Project Assessment

The final mini project shall require students to integrate the major DevOps practices covered in the laboratory. The project should demonstrate practical implementation of version control, collaborative development, Continuous Integration, containerization and/or automated configuration/provisioning, with proper execution evidence.

S.No.	Evaluation Component	Key Verification Points	Marks
1	Problem Definition & Project Planning	Problem statement, objectives, requirements, scope, module identification, technology selection and workflow planning.	10
2	Git Repository & Version Control	Repository creation, meaningful commits, README, .gitignore, remote repository, push/pull/fetch operations and commit history.	15
3	Branching, Merging & Collaboration	Feature branches, parallel development, merging, conflict handling, branch organization and collaborative workflow.	10
4	Application Development & Build	Working application/source code, project structure, dependency management, build configuration and successful execution.	15
5	Jenkins Continuous Integration	Jenkins configuration, source-code integration, automated checkout, build job/pipeline, build execution and console verification.	15
6	Docker / Containerization	Dockerfile, image creation, container execution, port/configuration handling, application deployment and container verification.	10
7	Automation / Configuration Management	Ansible/Chef/Puppet/SaltStack or equivalent automation where applicable; repeatable configuration/provisioning and successful execution.	5
8	Testing & Verification	Functional testing, build verification, error identification, test evidence and corrective action.	5
9	Documentation & Technical Presentation	Architecture/workflow diagram, commands/configuration, screenshots, results, troubleshooting, conclusion and repository details.	5
10	Final Demonstration & Viva-Voce	Live demonstration, explanation of implementation, troubleshooting responses and technical understanding.	10
		TOTAL	**100**

Recommended Mark Distribution within the Final Demonstration

For uniform evaluation, the examiner may use the following internal split for the implementation and demonstration components.

Assessment Area	Marks	Awarded Marks	Remarks
Working implementation and correctness	30		
DevOps tool integration and automation	20		
Version control and collaborative workflow	15		
Build / deployment / container verification	15		
Testing and troubleshooting	5		
Documentation and technical presentation	5		
Live demonstration and viva-voce	10		
TOTAL	**100**		

Minimum Evidence to be Submitted

- Project repository URL and project README.
- Git commit history and branch/merge evidence.
- Jenkins job/pipeline configuration and successful build evidence.
- Dockerfile, image and running container evidence where Docker is used.
- Automation/configuration scripts where applicable.
- Application execution and testing evidence.
- Architecture/workflow diagram.
- Short technical report containing implementation steps, results and troubleshooting.
- Live demonstration of the complete DevOps workflow.

Evaluation Guidelines

- Marks shall be awarded primarily for a working implementation and demonstrable output.
- Copied source code without functional understanding shall not receive full marks.
- Screenshots may support the evaluation, but critical components should be demonstrated live wherever possible.
- Students should use meaningful repository names, branch names and commit messages.
- If a component is not applicable to the selected project, the student shall obtain faculty approval and demonstrate an equivalent DevOps automation component.
- The final viva shall focus on the student's own implementation, commands used, tool configuration and troubleshooting.

Faculty Evaluation Record

Student Name		**Register No.**	
Project Title		**Date**	
Final Mark			
Faculty Evaluator I		**Signature**	
Faculty Evaluator II	/ 100	**Signature**	

TABLE OF CONTENTS

S. NO.	CONTENT	PAGE NO.
1	INTRODUCTION	05
2	OBJECTIVES	05
3	ABSTRACT	06
4	TECHOLOGY STACK	06
5	SYSTEM ARCHITECTURE	07
6	MODULE 1 — MONITORING USER INTERFACE & REAL-TIME DASHBOARD	08
7	MODULE 2 — FLASK BACKEND & METRIC COLLECTION ENGINE	09
8	MODULE 3 — SQLITE DATABASE MANAGEMENT	10
9	MODULE 4 — GIT & GITHUB SOURCE CONTROL	11
10	MODULE 5 — DOCKER CONTAINERIZATION	12
11	MODULE 6 — JENKINS CI/CD PIPELINE	14
12	MODULE 7 — AUTOMATED TESTING	15

13	MODULE 8 — ANSIBLE DEPLOYMENT	17w
14	MODULE 9 — PERSISTENT STORAGE	18
15	MODULE 10 — ANOMALY THRESHOLDS & AUTOMATED ALERT LIFECYCLE	19
16	MODULE 11 — CI/CD INTEGRATION	20
17	MODULE 12 — APPLICATION & DEPLOYMENT VERIFICATION	21
18	RESULTS AND OUTPUT	23
19	COMMANDS USED	23
20	ADVANTAGES	28
21	LIMITATIONS	28
22	FUTURE ENHANCEMENTS	29
23	CONCLUSION	29

1. INTRODUCTION

The Cloud Infrastructure Monitoring and Automated Alert System is a web-based observability platform developed to provide continuous visibility into computing resources. The system collects authentic CPU utilization, virtual memory consumption, filesystem disk usage, and system uptime using Python's psutil library. An operations dashboard provides real-time metric cards, historical trend charts, and automated incident alerting. The application is built using HTML5, CSS3, JavaScript, Python, Flask, and SQLite, with Flask orchestrating backend telemetry and SQLite storing metric snapshots, alert records, and configured thresholds. The project incorporates modern DevOps practices to automate the entire development, testing, and deployment lifecycle. Git and GitHub are used for distributed version control, Jenkins automates the declarative CI/CD pipeline, Docker containerizes the application on python:3.11-slim, Pytest performs automated testing with isolated mock databases, and Ansible automates container deployment and port mapping. A persistent Docker Volume ensures all database records remain intact across container lifecycles.

2.OBJECTIVES

- To develop a responsive cloud infrastructure monitoring dashboard that enables system administrators to observe CPU, memory, disk, and uptime metrics in real time.
- To implement an automated alert lifecycle management engine for detecting threshold breaches, deduplicating repetitive alerts, and automatically resolving incidents when metrics normalize.
- To organize system metrics and alert history systematically in an SQLite database (data/monitoring.db) using parameterized SQL queries.
- To implement DevOps practices throughout the application development and deployment process using Git, GitHub, and Jenkins.
- To automate the CI/CD workflow using Jenkins for building, testing, and deploying the application.
- To containerize the application using Docker to provide a consistent and portable execution environment.
- To integrate Pytest for automated testing to verify application functionality and reduce errors before deployment.
- To automate application deployment and container configuration using Ansible.
- To implement a named Docker Volume (cloud_monitoring_data) for persistent storage of the SQLite database, ensuring that operational history, alerts, and settings remain stored even when the container is recreated.

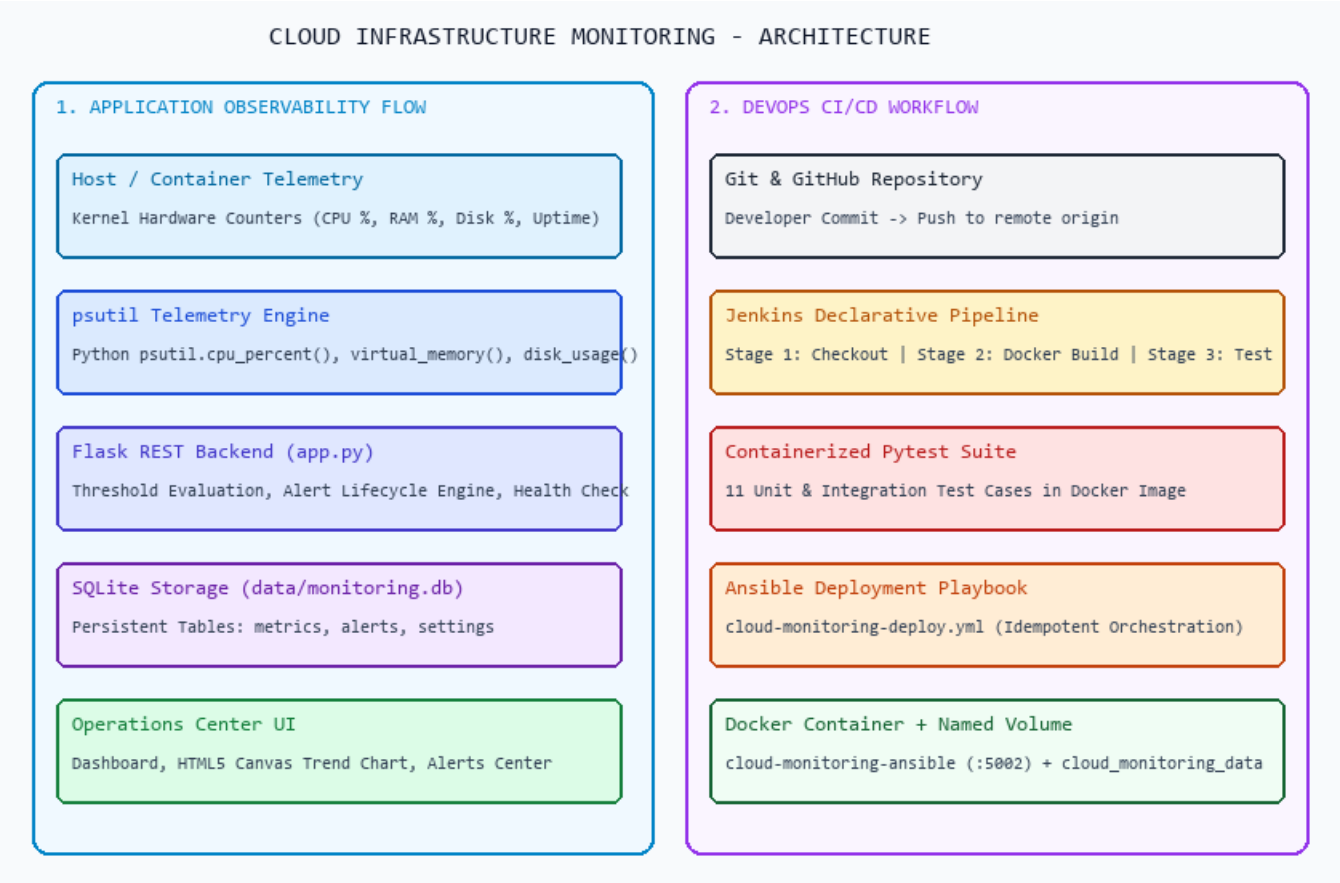
3. ABSTRACT

The Cloud Infrastructure Monitoring and Automated Alert System is a web-based observability platform developed to provide continuous visibility into computing resources. The system collects authentic CPU utilization, virtual memory consumption, filesystem disk usage, and system uptime using Python's psutil library. An operations dashboard provides real-time metric cards, historical trend charts, and automated incident alerting. The application is built using HTML5, CSS3, JavaScript, Python, Flask, and SQLite, with Flask orchestrating backend telemetry and SQLite storing metric snapshots, alert records, and configured thresholds. The project incorporates modern DevOps practices to automate the entire development, testing, and deployment lifecycle. Git and GitHub are used for distributed version control, Jenkins automates the declarative CI/CD pipeline, Docker containerizes the application on python:3.11-slim, Pytest performs automated testing with isolated mock databases, and Ansible automates container deployment and port mapping. A persistent Docker Volume ensures all database records remain intact across container lifecycles.

4.TECHNOLOGY STACK

Technology	Version	Purpose
HTML	HTML5	Web page structure
CSS	CSS3	User interface styling
JavaScript	ES6+	Frontend interaction
Python	3.11 / 3.14	Backend programming & telemetry engine
Flask	3.1.2	Backend WSGI framework & REST API
SQLite	3.x	Relational database storage (metrics, alerts, settings)
Git	2.x	Version control
GitHub	—	Source code repository
Jenkins	2.541.3	CI/CD automation
Docker	28.4.0	Application containerization & health checks
Pytest	9.1.1	Automated unit & integration testing (11 tests)
Ansible	2.10.8	Deployment automation
Docker SDK / Community Docker	community.docker 1.2.2	Docker management through Ansible

5.SYSTEM ARCHITECTURE



1. Development: Developer creates and updates the Cloud Infrastructure Monitoring application.
2. Source Code Management: The application source code and configuration files are maintained using Git.
3. GitHub Repository: The updated code is pushed to the GitHub repository for centralized source code management.
4. Jenkins Trigger: Jenkins retrieves the latest source code from the GitHub repository and starts the CI/CD pipeline.
5. Docker Build: Jenkins builds the application into a Docker image containing the Flask application and required dependencies.
6. Automated Testing: Jenkins runs Pytest to verify the application before deployment.
7. Deployment Automation: After successful testing, Jenkins invokes Ansible to automate the deployment process.
8. Container Configuration: Ansible creates and configures the Docker container with the required image, port mapping, restart policy, and volume configuration.
9. Application Execution: The Docker container runs the Flask backend, HTML/CSS/JavaScript frontend, and SQLite database.

10. User Interaction: System administrators view real-time metric cards, inspect historical trend charts, and configure alert thresholds.
11. Alert Management: The automated alert engine evaluates metrics against thresholds, deduplicates sustained anomalies, and auto-resolves cleared incidents.

MODULE 1 — MONITORING USER INTERFACE & REAL-TIME DASHBOARD

This module provides the operations dashboard interface for the Cloud Infrastructure Monitoring application. It is designed to make system resource observation, anomaly detection, and historical review clear and immediate.

Main Features

- Displays real-time metric cards for CPU, Memory (RAM), and Disk Storage with progress bars.
- Displays system overview metadata including Global Health status, Monitored Node name, and Uptime.
- Features an active alert notification strip highlighting unhandled infrastructure incidents.
- Provides an automated 5-second AJAX polling mechanism with countdown indicator.
- Provides a manual 'Refresh Now' trigger button for instant on-demand metric collection.
- Presents a recent metric snapshots table displaying the last 10 records stored in SQLite.
- Renders an interactive multi-line resource utilization trend chart using plain HTML5 Canvas.
- Allows administrators to interactively navigate between Dashboard, History, Alerts, and Settings.
- Provides clear visual cues (green for normal, red/orange for high load) based on configured limits.
- Delivers a modern, dark cloud-operations aesthetic optimized for laboratory demonstration.
- Uses responsive interface elements for better usability.
- Connects the user interface with the Flask backend for processing requests.

Technologies: HTML, CSS, JavaScript.

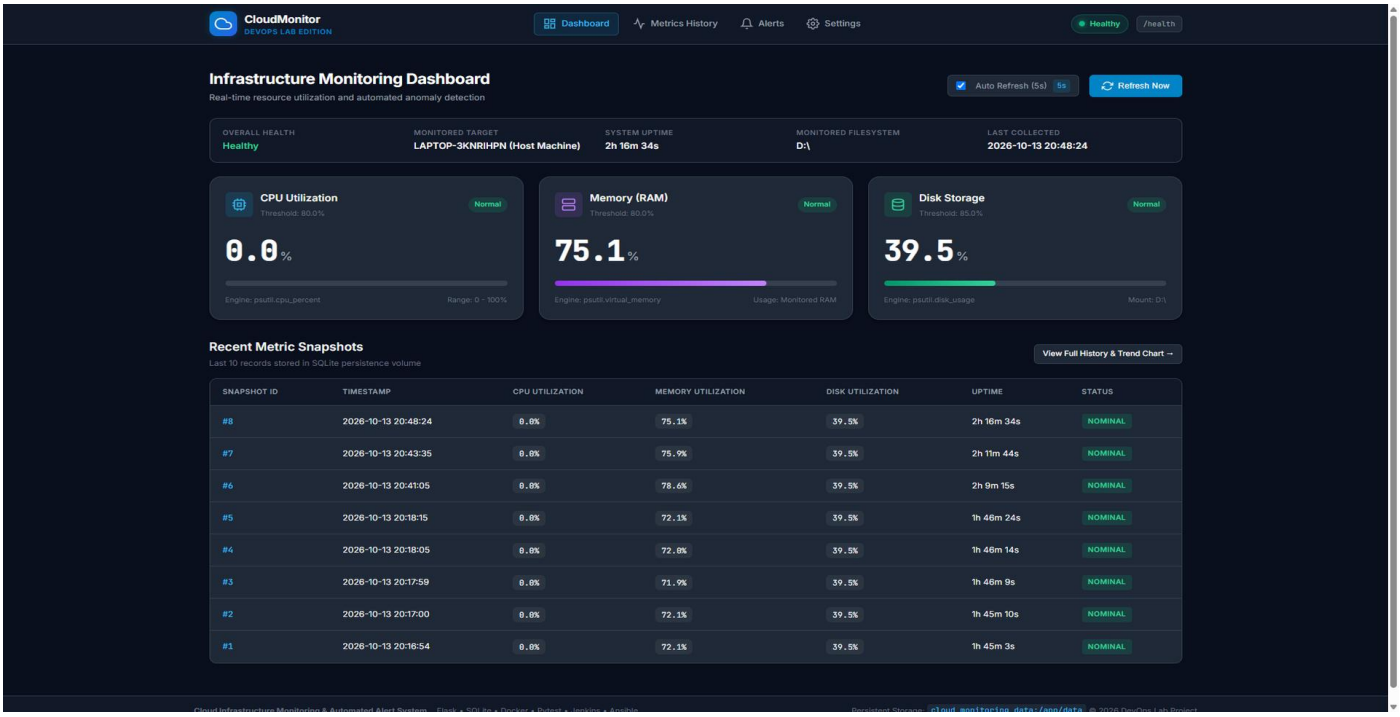


Figure 7.1: Cloud Infrastructure Monitoring Dashboard Overview

Figure 7.2: Metric Cards and Real-Time Utilization Progress Meters

MODULE 2 — FLASK BACKEND & METRIC COLLECTION ENGINE

This module handles the application logic and acts as the communication layer between the frontend and the operating system kernel. Flask processes REST requests and manages live telemetry collection via Python's psutil library.

Main Functions

- Receives requests from the frontend.
- Collects authentic kernel metrics using psutil.cpu_percent(), psutil.virtual_memory(), and psutil.disk_usage().
- Serves primary dashboard view at /dashboard with real-time metrics and active incident summaries.
- Exposes REST API endpoint /metrics returning structured JSON telemetry for automated collectors.
- Provides metrics history endpoint /api/history for client-side HTML5 Canvas trend charting.
- Evaluates current metrics against configured thresholds and coordinates alert lifecycle actions.
- Records metric snapshots and incident alerts into SQLite with thread-safe parameterized queries.
- Provides container health check endpoint at /health returning operational and database status.
- Handles requests related to the admin dashboard.
- Connects the frontend interface with backend services.
- Processes application routes and requests.

- Runs the Flask application inside the Docker container.
- Provides the backend telemetry services required by the Cloud Infrastructure Monitoring application.

Technology: Python with Flask.

Flask Backend Engine - app.py & monitoring.py

```
$ cat app.py
"""Cloud Infrastructure Monitoring - Flask Controller"""
from flask import Flask, render_template, request, jsonify
import monitoring

app = Flask(__name__)
monitoring.init_db()  # Initialize SQLite schema

@app.route('/dashboard')
def dashboard():
    metrics_data = monitoring.get_system_metrics()
    monitoring.evaluate_and_record_metrics(metrics_data)
    active_alerts = monitoring.get_alerts(status_filter='Active')
    return render_template('index.html', metrics=metrics_data, alerts=active_alerts)

@app.route('/metrics', methods=['GET'])
def metrics_endpoint():
    m = monitoring.get_system_metrics()  # Real psutil reading
    s_id, actions = monitoring.evaluate_and_record_metrics(m)
    return jsonify({'status': 'success', 'snapshot_id': s_id, 'metrics': m})
```

Figure 7.3: Flask Backend Implementation (app.py & monitoring.py)

MODULE 3 — SQLITE DATABASE MANAGEMENT

This module manages the storage and retrieval of infrastructure metric snapshots, incident alerts, and anomaly settings using SQLite. The database provides persistent application data that can be accessed by the Flask backend.

Main Functions

- Stores periodic metric snapshots in the 'metrics' table (CPU %, RAM %, Disk %, Uptime, timestamp).
- Maintains automated alert lifecycle logs in the 'alerts' table (metric_name, value, threshold, severity, status).
- Stores administrator-configurable thresholds in the 'settings' table (cpu_threshold, memory_threshold, disk_threshold).
- Provides database access to the Flask backend.
- Provides chronological query filtering with configurable snapshot limits (20, 50, 100).
- Makes alert records accessible to the Incident Alert Center with Active/Resolved status filters.
- Maintains the monitoring.db database file located in the data/ directory.

- Stores the database under the application data directory.
- Works with the Docker Volume for persistent storage.
- Provides structured storage for telemetry snapshots and incident alerts.
- Maintains metric history and alert records even when the application container is recreated, through the configured Docker Volume.

Database Location

data/monitoring.db

Inside the Docker container:

data/monitoring.db

Technology: SQLite.

Figure 7.4: SQLite Database Telemetry Records (metrics, alerts, settings)

MODULE 4 — GIT & GITHUB SOURCE CONTROL

development history of the Cloud Infrastructure Monitoring project. Git is used locally for version control, while GitHub provides the remote repository.

Main Functions

- Initializes the project as a Git repository.
- Tracks changes made to project files.
- Creates commits for project updates.
- Maintains the history of project development.
- Allows changes to be reviewed through Git history.
- Pushes project code to GitHub.
- Provides a centralized remote repository.
- Allows Jenkins to retrieve the latest source code.
- Maintains the important project files in one repository.
- Supports integration between source control and the Jenkins pipeline.
- Helps maintain different versions of the project.

Main Project Files

app.py

Dockerfile

Jenkinsfile

test_app.py

requirements.txt

.gitignore

Technologies: Git and GitHub.

Figure 7.5: Cloud Infrastructure Monitoring Git Repository & Commit History

MODULE 5 — DOCKER CONTAINERIZATION

Docker is used to package and run the Cloud Infrastructure Monitoring application in an isolated and consistent environment. The application and its required dependencies are packaged into a Docker image.

Main Functions

- Creates a Docker image from the application.
- Packages the application and required dependencies.
- Provides an isolated execution environment.
- Creates the application container from the Docker image.
- Runs the Flask application inside the container.
- Provides port mapping for application access.
- Provides a consistent environment for testing and deployment.
- Allows the application to run independently of the host environment.
- Works with Jenkins for automated image creation.
- Works with Ansible for automated deployment.
- Provides a repeatable application deployment environment.

Docker Configuration

Docker Image:

cloud-monitoring-ci

Docker Container:

cloud-monitoring-ansible

Port Mapping:

5002 → 5000

The application can be accessed through:

http://localhost:5002

Technology: Docker.

Docker Flow

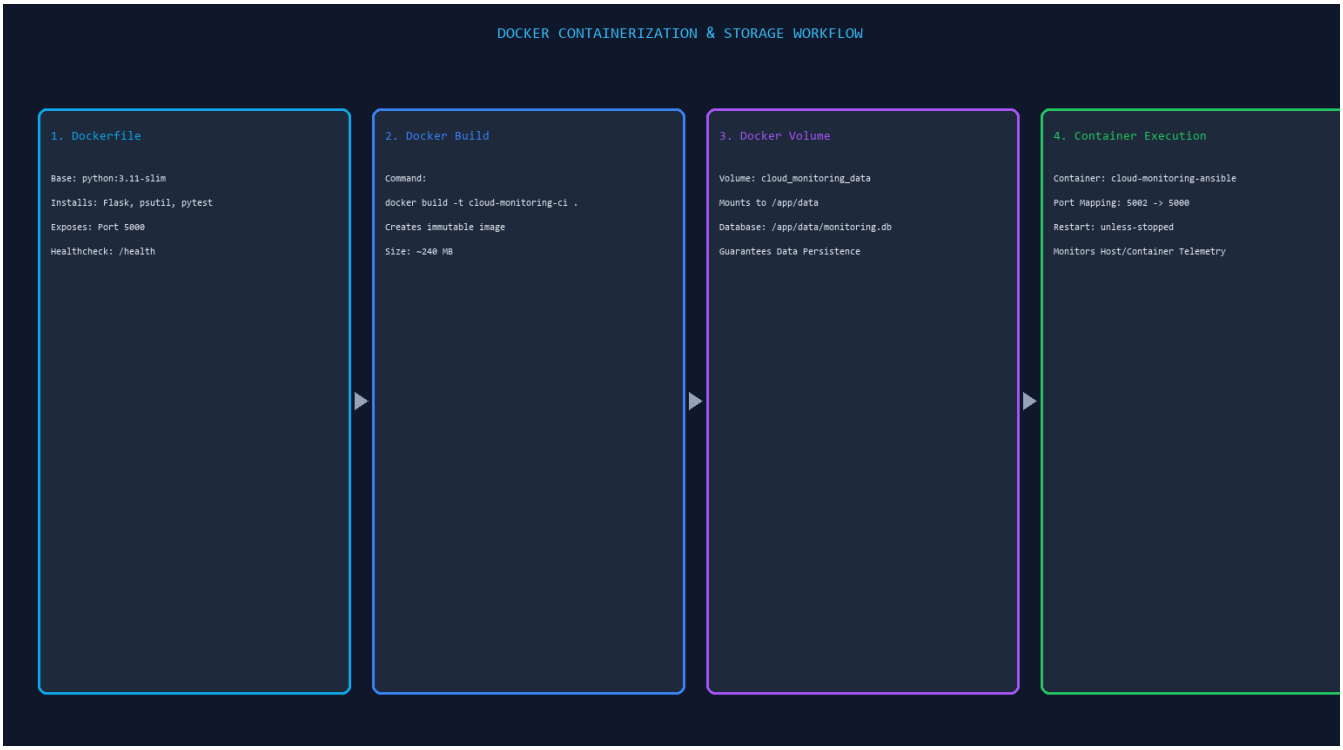


Figure 7.6: Cloud Monitoring Docker Image (cloud-monitoring-ci)

Figure 7.7: Running Docker Container (cloud-monitoring-ansible)

MODULE 6 — JENKINS CI/CD PIPELINE

Jenkins is used to automate the Continuous Integration and Continuous Deployment process. It connects the source code repository with the application build, testing, and deployment stages.

Main Functions

- Retrieves the latest source code from GitHub.
- Checks out the required project version.
- Builds the Docker image.
- Executes automated tests.
- Provides a sequence of automated pipeline stages.
- Prevents later stages from proceeding when an earlier stage fails.
- Invokes the Ansible playbook after successful testing.

- Provides pipeline execution logs.
- Reports pipeline success or failure.
- Automates the overall CI/CD process.
- Reduces repetitive manual build and deployment activities.

Figure 7.8: Jenkins CI/CD 4-Stage Declarative Pipeline

Figure 7.9: Successful Jenkins Pipeline Console Execution Output

MODULE 7 — AUTOMATED TESTING

This module validates the application automatically before deployment. Pytest is integrated into the Jenkins pipeline so that the application is tested before the deployment stage.

Main Functions

- Uses Pytest for automated testing.
- Maintains the test file test_app.py.
- Executes the test inside the Docker image.
- Verifies the configured application test.
- Provides automated test execution.
- Prevents deployment from continuing when an earlier pipeline stage fails.
- Displays test results in Jenkins.
- Integrates testing directly into the CI/CD workflow.
- Reduces the need for repeated manual testing.
- Helps identify application issues before deployment.

Test File

test_app.py

Test Command

python -m pytest test_app.py

Test Result

collected 11 items
test_app.py ..... [100%]
11 passed in 0.43s

test_app.py . [100%]

11 passed in 0.43s [100%]

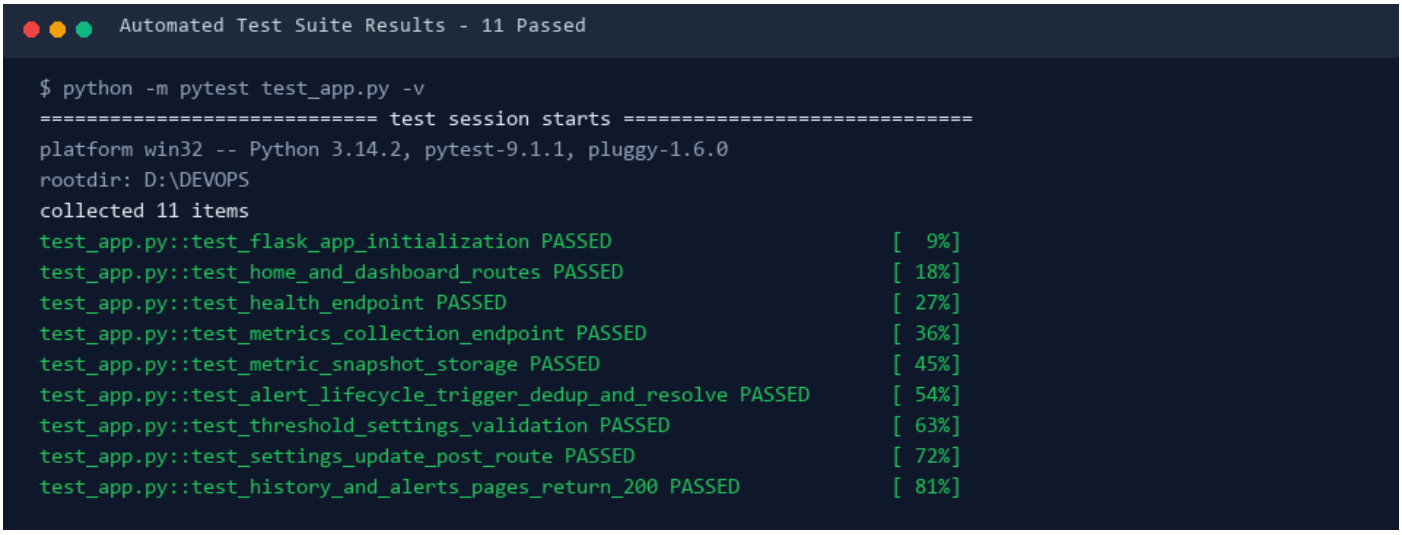


Figure 7.10: Successful Automated Test Execution (11 Tests Passed)

MODULE 8 — ANSIBLE DEPLOYMENT

Ansible is used to automate the deployment and configuration of the Cloud Infrastructure Monitoring Docker container. The deployment configuration is defined in the Ansible playbook cloud-monitoring-deploy.yml.

Main Functions

- Executes the deployment playbook.
- Specifies the Docker container name.
- Specifies the Docker image.
- Configures application port mapping.
- Configures persistent volume mounting.
- Defines the container state.
- Defines the restart policy.
- Uses Docker to apply the specified configuration.
- Automates container deployment.
- Allows deployment to be triggered through Jenkins.
- Provides a repeatable deployment configuration.

Deployment Configuration

cloud-monitoring-ansible

cloud-monitoring-ci

published_ports:

published_ports:
- "5002:5000"

volumes:

volumes:
- "cloud_monitoring_data:/app/data"

Ansible Playbook - cloud-monitoring-deploy.yml

```
$ cat cloud-monitoring-deploy.yml
---
- name: Deploy Cloud Infrastructure Monitoring Container
  hosts: all
  become: false
  vars:
    container_name: 'cloud-monitoring-ansible'
    image_name: 'cloud-monitoring-ci'
    host_port: '5002'
    volume_name: 'cloud_monitoring_data'
  tasks:
    - name: 1. Ensure persistent Docker volume exists
      community.docker.docker_volume:
        name: '{{ volume_name }}'
    - name: 2. Deploy or update Docker container
      community.docker.docker_container:
```

Ansible Deployment Output - Idempotent Execution

```
$ ansible-playbook -i inventory.ini cloud-monitoring-deploy.yml
PLAY [Deploy Cloud Infrastructure Monitoring Container] ************************
TASK [1. Display deployment parameters] ***************************************
ok: [localhost] => { 'msg': 'Deploying cloud-monitoring-ansible using image cloud-monitoring-ci' }
TASK [2. Ensure persistent Docker volume exists] ******************************
ok: [localhost] (state: present)
TASK [3. Deploy or update Cloud Monitoring Docker container] ******************
changed: [localhost] (container started, port 5002:5000, volume mounted)
TASK [4. Display container deployment result] *********************************
ok: [localhost] => { 'msg': 'Container deployed successfully! Access: http://localhost:5002/dashboard' }
PLAY RECAP ********************************************************************
```

Figure 7.11: Ansible Deployment Playbook Execution Output

MODULE 9 — PERSISTENT STORAGE

This module ensures that database information remains available even when the application container is recreated. Docker Volume is used to separate persistent application data from the container lifecycle.

Main Functions

- Creates persistent storage for application data.
- Uses the Docker Volume cloud_monitoring_data.
- Mounts the volume to /app/data.

- Stores the SQLite database in the mounted directory.
- Separates persistent data from the container lifecycle.
- Allows the database to remain available after container recreation.
- Preserves stored metric history, incident alerts, and configured thresholds.
- Provides persistent storage for the SQLite database.
- Allows the application container to be recreated without losing stored metric or alert data.

Storage Configuration

cloud_monitoring_data:/app/data

Storage Flow

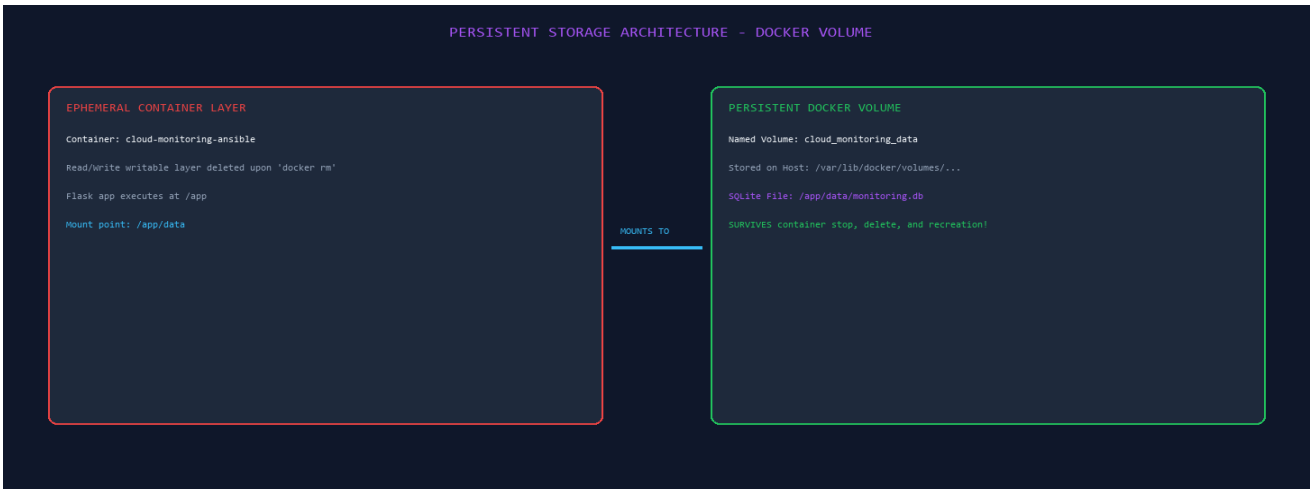


Figure 7.12: Cloud Monitoring Docker Volume (cloud_monitoring_data)

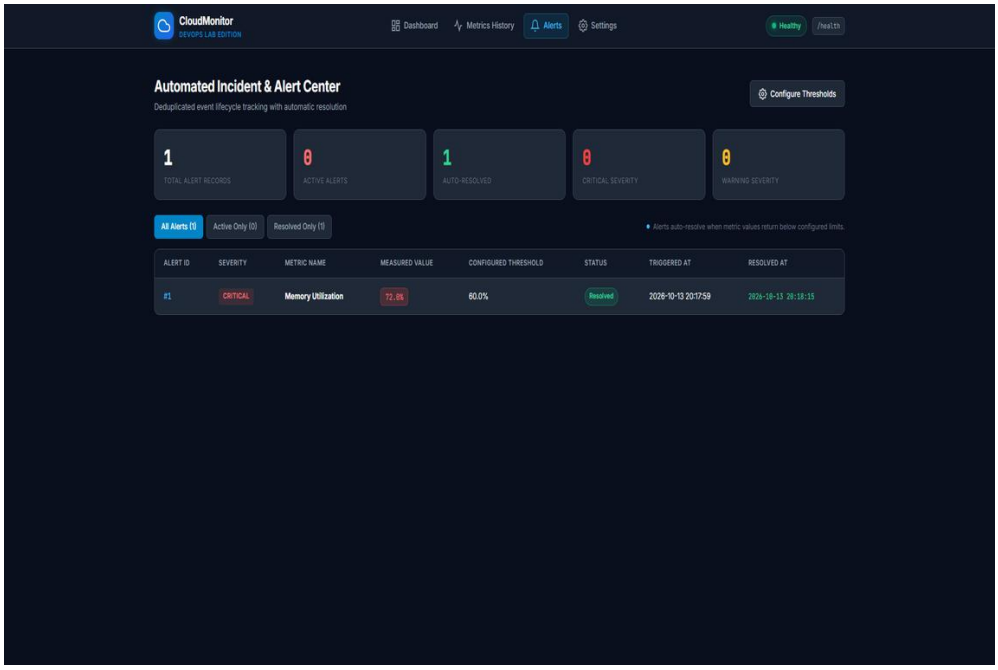
MODULE 10 — ANOMALY THRESHOLDS & AUTOMATED ALERT LIFECYCLE

This module manages the automated alert lifecycle and dynamic anomaly threshold evaluation. The Flask backend compares incoming psutil metrics against configured limits, while the Alerts Center provides real-time lifecycle tracking.

Main Functions

- Evaluates CPU, RAM, and Disk metrics against dynamic thresholds.
- Creates active incident alerts when measured telemetry exceeds configured limits.
- Implements intelligent deduplication to update active alerts instead of spamming duplicate rows.
- Assigns severity levels (Warning vs Critical) based on degree of threshold breach.
- Automatically resolves active alerts when metric values return below configured limits.
- Records exact resolved_at timestamps when incidents are automatically resolved.
- Provides an interactive Settings interface allowing administrators to configure thresholds (1-100%).

- Persists updated threshold limits in the settings table of data/monitoring.db.
- Displays incident statistics (Total, Active, Resolved, Critical, Warning) in the Alerts Center.



Admin Dashboard

http://localhost:5002

Figure 7.13: Automated Incident Alert Center & Deduplication (/alerts)

MODULE 11 — CI/CD INTEGRATION

This module integrates the different DevOps tools into a single automated workflow. GitHub, Jenkins, Docker, Pytest, and Ansible work together to automate the build, testing, and deployment process.

Main Functions

- Integrates GitHub with Jenkins.
- Retrieves the latest project code.
- Checks out the required project version.
- Builds the Docker image automatically.
- Runs Pytest automatically.
- Triggers Ansible deployment after successful testing.
- Creates and manages the Docker container.
- Provides a repeatable deployment process.
- Produces execution results for each pipeline stage.
- Reduces repetitive manual deployment activities.

- Connects source control, testing, containerization, and deployment into one workflow.

Figure 7.14: Complete CI/CD Pipeline Execution & Verification

MODULE 12 — APPLICATION & DEPLOYMENT VERIFICATION

This module verifies that the complete Cloud Infrastructure Monitoring system is functioning correctly after deployment. Verification is performed across the application, Docker container, database, storage, Ansible deployment, and Jenkins pipeline.

Verification Activities

- Checking the running Docker container.
- Checking the application port.
- Opening the Cloud Infrastructure Monitoring dashboard.
- Observing real-time CPU, RAM, and Disk percentage cards on the dashboard.
- Simulating alert generation by lowering threshold below active machine load.
- Verifying alert creation, deduplication across multiple polls, and auto-resolution.
- Inspecting historical metrics log and HTML5 Canvas trend visualization.
- Querying metrics and alerts tables in SQLite (/app/data/monitoring.db).
- Checking the Docker Volume.
- Checking Ansible execution.
- Checking Jenkins pipeline results.
- Confirming successful application deployment.
- Verifying that telemetry history and alert records survive container removal and recreation.

Important Outputs

Docker Build → SUCCESS

Pytest → 11 PASSED in 0.43s

Ansible → failed=0

Pipeline → SUCCESS

Figure 7.15: Successfully Deployed Cloud Infrastructure Monitoring Web Application

19. RESULTS AND OUTPUT

The implemented system successfully provides:

- Real-time host and container resource telemetry (CPU, RAM, Disk, Uptime)
- Automated 5-second AJAX polling with countdown indicator and manual refresh
- Configurable anomaly thresholds with strict numeric validation (1.0% to 100.0%)
- Automated incident alert creation, deduplication, and automatic resolution
- Historical metric snapshot logging with HTML5 Canvas trend line chart
- Container health check endpoint (/health) for monitoring container liveliness
- Responsive operations dashboard, metrics history view, alerts center, and settings view
- SQLite database storage
- Docker containerization
- Automated Pytest testing
- Jenkins CI/CD automation
- Ansible deployment
- Persistent Docker Volume storage (cloud_monitoring_data mounted to /app/data)

COMMANDS USED

1. Docker Environment

Check running containers

```
docker ps
```

Check all containers

```
docker ps -a
```

Start Ansible container

```
docker start ansible-docker-lab
```

Start Jenkins

```
docker start jenkins
```

2. Jenkins Commands

Fix Docker socket permission for Jenkins

docker exec -u root jenkins chmod 666 /var/run/docker.sock

Verify Jenkins can access Docker

docker exec jenkins docker info

Check Jenkins port

docker port jenkins

Jenkins was accessed using:

http://localhost:9091

3. Docker Image Build

Jenkins executes:

cloud-monitoring-ci

This creates the Docker image:

cloud-monitoring-ci

4. Automated Testing

Jenkins executes:

cloud-monitoring-ci

Expected result:

11 passed in 0.43s [100%]

5. Ansible Deployment

Jenkins executes:

docker exec ansible-docker-lab ansible-playbook -i /inventory.ini /docker-deploy.yml

This runs the Ansible playbook.

Run Ansible manually

docker exec ansible-docker-lab ansible-playbook -i /inventory.ini /docker-deploy.yml

View Ansible playbook

docker exec ansible-docker-lab cat /docker-deploy.yml

6. Docker Container Verification

Check deployed container

docker ps

The Cloud Monitoring container:

cloud-monitoring-ansible

Port mapping:

5001:5000

Application:

http://localhost:5002

7. Docker Volume Commands

List Docker volumes

docker volume ls

Our volume:

cloud_monitoring_data

Inspect the volume

cloud_monitoring_data

8. SQLite Database Commands

Display stored telemetry metrics in SQLite

data/monitoring.db

Display database tables

data/monitoring.db

Display stored incident alerts in SQLite

data/monitoring.db

9. Docker Volume + Database Verification

To verify that the database exists inside the running container:

data/monitoring.db

The database location is:

data/monitoring.db

and inside the container:

data/monitoring.db

10. Git Commands Used

Initialize Git

git init

Check status

git status

Add files

git add .

Commit changes

Finalize Cloud Infrastructure Monitoring and Automated Alert System

Check branches

git branch

Connect GitHub repository

https://github.com/DevOps-Lab/CloudInfrastructureMonitoring.git

Push to GitHub

git push -u origin master

Pull latest code

git pull origin master

11. Useful Docker Commands Used During Implementation

List Docker images

docker images

Stop container

cloud-monitoring-ansible

Start container

cloud-monitoring-ansible

View container logs

cloud-monitoring-ansible

Execute a command inside the container

cloud-monitoring-ansible

12. Main Commands for Your Report

If your report doesn't need every troubleshooting command, **these are the most important ones to include**:

cloud-monitoring-ci
cloud-monitoring-ci
docker exec ansible-docker-lab ansible-playbook -i /inventory.ini /docker-deploy.yml
docker ps
docker volume ls
cloud_monitoring_data
data/monitoring.db

And the **Git commands**:

git add .
Finalize Cloud Infrastructure Monitoring and Automated Alert System
git push -u origin master

21. ADVANTAGES

- Provides real-time host and container observability with zero external SaaS subscription fees.
- Eliminates alert fatigue through intelligent incident deduplication during sustained high load.
- Ensures hands-off operational management with automatic alert resolution when metrics normalize.
- Automates application testing.
- Reduces manual deployment effort.
- Provides consistent application execution through Docker.
- Guarantees zero data loss across container teardown or upgrade using named Docker Volumes.
- Integrates source control with CI/CD.
- Demonstrates automated deployment using Ansible.

22. LIMITATIONS

- Uses embedded SQLite storage, suitable for single-node deployments rather than distributed multi-master clusters.
- Container metric collection observes the container's visible cgroups boundaries rather than physical bare-metal hardware.
- Alert notifications are presented in-app; real-time SMS/Email dispatch requires external SMTP or Twilio API keys.
- Does not monitor third-party public cloud accounts (AWS/Azure) unless external cloud API credentials are provided.
- The deployment is demonstrated in a local Docker environment.
- Long-term historical trend retention is managed by SQLite rather than a distributed time-series database like Prometheus.

23. CONCLUSION

cloud_monitoring_data

24. FUTURE ENHANCEMENTS

The system can be extended with:

1. Distributed multi-node agent network reporting telemetry to a central aggregator.
2. External public cloud integrations (AWS CloudWatch, Azure Monitor, GCP Cloud Monitoring).
3. Automated alert webhooks dispatching notifications to Slack, Discord, or Microsoft Teams.
4. Prometheus metrics exporter endpoint (/metrics/prometheus) and Grafana dashboard templates.
5. PostgreSQL or MySQL clustering for high-throughput enterprise environments.
6. Role-based access control (RBAC) and administrator authentication.
7. Cloud deployment.
8. PostgreSQL or MySQL integration.
9. GitHub webhook-based automatic triggering.
10. Application monitoring and centralized logging.